

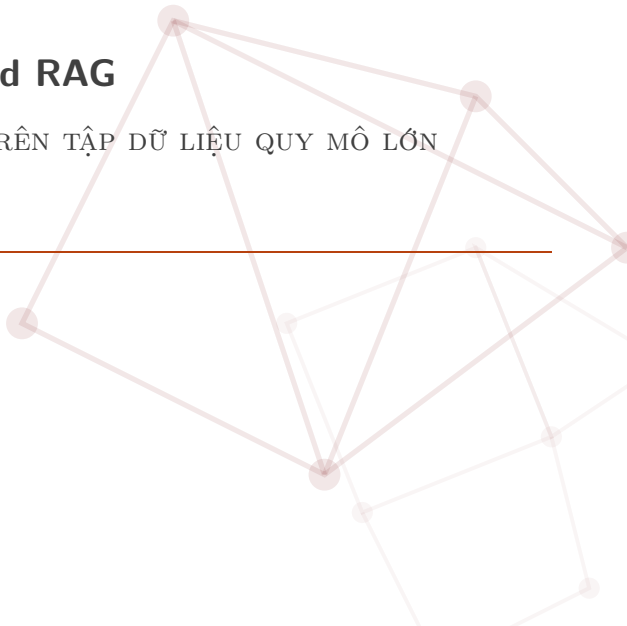
T-RAG: Temporal & Targeted RAG

KHẮC PHỤC HẠN CHẾ CỦA RAG TRÊN TẬP DỮ LIỆU QUY MÔ LỚN
(ENTERPRISE RAG-BENCH)

Người trình bày: Bảo & Bằng

Hệ thống RAG Benchmark & Research

Ngày 26 tháng 6 năm 2026



Bối cảnh & Thách thức

Kiến trúc Đề xuất: T-RAG Pipeline

Thiết lập Môi trường & Đo lường (Benchmark Setup)

Bối cảnh & Thách thức



Đặc tả tập dữ liệu EnterpriseRAG-Bench

Tính chất của dữ liệu thực tế

Khác với Wikipedia, tập dữ liệu nội bộ bao gồm nhiều định dạng đa dạng, cấu trúc không đồng nhất và có tần suất cập nhật cao.

Slack

(285k docs)

Hội thoại ngắn

Gmail

(121k docs)

Luồng email (Thread)

Linear/Jira

(41k docs)

Tiến độ dự án

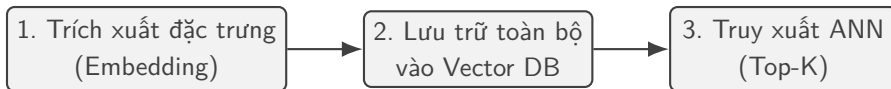
Confluence

(5k docs)

Văn bản cấu trúc dài

Quy mô tập dữ liệu: Hơn **500.000 văn bản** từ 9 nguồn dữ liệu khác nhau.

Phân tích phương pháp RAG Tiêu chuẩn (Standard RAG)



Các nguyên nhân dẫn đến suy giảm hiệu suất:

- ❌ **Mật độ Vector cao:** Với 500.000 tài liệu, khoảng cách Cosine giữa các vector trở nên kém phân biệt. Nhiều tài liệu chung chủ đề dễ bị truy xuất nhầm.
- ❌ **Thiếu nhận thức thời gian (Temporal Blindness):** Tương đồng ngữ nghĩa không phản ánh tính mới của thông tin (ví dụ: Quyết định năm 2022 so với 2024).
- ❌ **Bất đồng từ vựng (Vocabulary Mismatch):** Truy vấn của người dùng thường mang tính ngữ cảnh (contextual), trong khi văn bản gốc chứa mã định danh hoặc tên kỹ thuật.

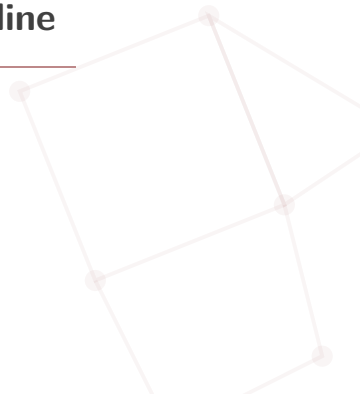
Giải pháp đề xuất: Mô hình T-RAG

✓ Phương pháp tiếp cận

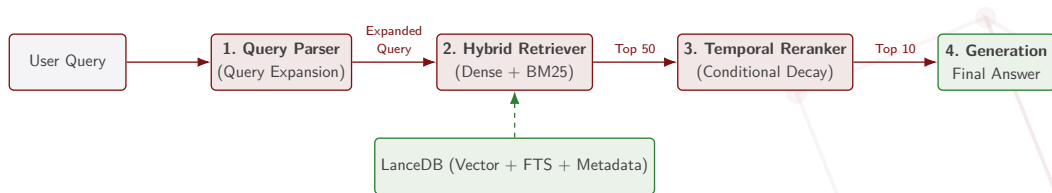
T-RAG đề xuất kiến trúc xử lý tuần tự, tích hợp tiền xử lý có định hướng (**Targeted**) và xếp hạng theo trục thời gian (**Temporal**).

- **Tiền lọc Metadata (Pre-filtering):** Sử dụng cơ sở dữ liệu quan hệ (SQL) để thu hẹp không gian tìm kiếm, loại bỏ sớm các tài liệu không liên quan trước khi tính toán độ tương đồng.
- **Phân bổ trọng số thời gian (Time Decay):** Áp dụng hàm suy giảm để giảm trọng số các tài liệu cũ, đảm bảo tính cập nhật của thông tin truy xuất.
- **Truy xuất lai (Hybrid Retrieval):** Kết hợp kỹ thuật Sparse (BM25) và Dense (Embedding) nhằm tối ưu khả năng đối khớp từ vựng và ngữ nghĩa.

Kiến trúc Đề xuất: T-RAG Pipeline



Cấu trúc Tổng quan T-RAG



Phân hệ 1: Hợp nhất Lưu trữ (Unified Storage với LanceDB)

Cơ sở dữ liệu hợp nhất (Unified Database)

Kiến trúc T-RAG đồng nhất hóa quy trình quản lý dữ liệu thông qua LanceDB, loại bỏ độ trễ và sự phân mảnh của kiến trúc đa cơ sở dữ liệu.

- **Dense Vector Index:** Chuyên biệt hóa cho biểu diễn nhúng (Embedding) nhằm phục vụ tìm kiếm ngữ nghĩa (Semantic Search).
- **Sparse FTS Index (Tantivy):** Tích hợp cơ chế tìm kiếm toàn văn bản (Full-Text Search) trực tiếp trên cùng một phân vùng dữ liệu.
- **SQL Metadata Filtering:** Hỗ trợ truy vấn siêu dữ liệu (source_type, thời gian) với khả năng tích hợp tự động vào thuật toán lai (RRF).
- **Tối ưu hiệu năng:** Triệt tiêu hoàn toàn nút thắt cổ chai I/O (I/O Bottleneck) khi đồng bộ hóa dữ liệu.

Phân hệ 2: Tự động Mở rộng & Trích xuất (Self-Query Expansion)

✂ Bộ tiền xử lý ngôn ngữ (LLM-based Query Parser)

Ứng dụng Mô hình Ngôn ngữ Lớn để phân tích truy vấn, trích xuất siêu dữ liệu và tự động sinh truy vấn tối ưu (HyDE - Hypothetical Document Embeddings).

Quá trình xử lý truy vấn:

- **Trích xuất siêu dữ liệu định hướng (Targeted Metadata):** Phân loại và nhận diện nguồn tài liệu tương ứng (Ví dụ: github, gmail).
- 🕒 **Nhận diện yếu tố thời gian (Temporal Intent):** Kích hoạt cờ `requires_latest=True` đối với các truy vấn đòi hỏi thông tin cập nhật, nhằm giảm thiểu sai số đối với các câu hỏi mang tính lịch sử.
- 🔍 **Mở rộng truy vấn (Query Expansion):** Tái cấu trúc truy vấn, bổ sung từ đồng nghĩa và diễn giải từ viết tắt, góp phần nâng cao độ phủ (Recall) trong tìm kiếm lại.

Phân hệ 3: Truy xuất Lai (Hybrid Retriever)

🔍 Tích hợp Dense & Sparse

Hoạt động truy xuất chỉ được thực hiện trên tập ứng viên giới hạn (Candidate List) thay vì toàn bộ cơ sở dữ liệu.

- **Dense Search (Semantic):** Đánh giá độ tương đồng Cosine, cho phép ánh xạ các khái niệm tương đương dù không trùng khớp từ vựng.
- **Sparse Search (Lexical - BM25):** Cần thiết để nhận diện chính xác các mã định danh, ID đặc thù (như mã lỗi, số ticket) mà Dense Search thường bỏ sót.
- **Hợp nhất bằng RRF (Reciprocal Rank Fusion):** Kết hợp thứ hạng từ cả hai phương pháp để trích xuất tập tài liệu liên quan nhất.

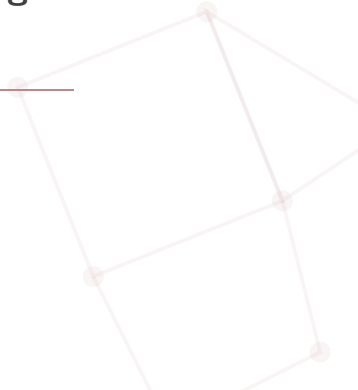
Phân hệ 4: Mô hình Reranker (Conditional Time Decay)

📊 Hàm suy giảm thời gian (Time Decay Function)

$$\text{Final_Score} = \text{Relevance} \times e^{-\lambda \Delta t} \quad (1)$$

- **Relevance** ($0 \rightarrow 1$): Điểm số tương đồng ngữ nghĩa trích xuất từ mạng Cross-Encoder.
- **Hệ số suy giảm có điều kiện (Conditional Time Penalty):**
 - Khi `requires_latest = True`: $\lambda > 0$, áp dụng suy giảm hàm mũ đối với các tài liệu cũ.
 - Khi `requires_latest = False`: $\lambda = 0$ (Hệ số = 1.0), bảo toàn tính toàn vẹn của kết quả truy xuất cho các truy vấn tra cứu sự kiện lịch sử.
- **Giới hạn ngữ cảnh (Context Truncation):** Thuật toán tự động cắt tỉa văn bản nhằm ngăn chặn hiện tượng tràn bộ nhớ (Out-of-Memory) khi chiều dài ngữ cảnh vượt ngưỡng cho phép của LLM.

Thiết lập Môi trường & Đo lường (Benchmark Setup)



Mô hình Triển khai: Kiến trúc Stage-Based Batching

🖥️ **Phần cứng:** 1 Node GPU H100 (40GB - 80GB VRAM).

🏆 **Mục tiêu Đánh giá:** Tối đa hóa thông lượng (Throughput) trên 500k tài liệu thuộc EnterpriseRAG-Bench.

🔧 Kiến trúc xử lý lô (Batching Architecture):

- Giải quyết độ trễ mạng (Network Overhead) bằng phương pháp xử lý lô đồng loạt cho từng luồng (Stage).
- Khai thác tối đa kỹ thuật *PagedAttention* của **vLLM**, gia tăng khả năng lấp đầy VRAM và tăng tốc độ sinh văn bản (Generation Speed).

⚙️ Quy hoạch Tài nguyên Cục bộ (Local Resource Allocation)

- **LLM Engine (vLLM Offline Batching):** Vận hành Llama-3-8B (Yêu cầu ~ 16GB). Đảm nhiệm đồng thời *Query Expansion* và *Answer Generation*.
- **Mạng Cross-Encoder:** Vận hành BGE-Reranker thông qua Sentence-Transformers (~ 4GB). Áp dụng Batch-size lớn nhằm tối ưu hóa thời gian tính toán ma trận tương đồng.
- **Cơ sở dữ liệu Hợp nhất (Unified LanceDB):**
 - Lưu trữ ngoại vi (In-memory / SSD) nhằm giải phóng bộ nhớ GPU.
 - Tận dụng kiến trúc backend Rust để tối ưu thời gian phản hồi cho các truy vấn Hybrid Search.



Thank you!

Hệ thống T-RAG

